LAB 7

Fuzz Testing for Input Validation Bugs

Hypothesis-Based Property Testing

Student Details	
Student Name	Anushka P
SRN	PES1UG24CS075
Section	B
Date	11 November 2026

1. Objective

The objective of this experiment is to explore fuzz testing as a software quality and security testing technique. The provided Python electricity-bill application is tested with automatically generated and unexpected inputs using the Hypothesis library. The aim is to identify input-validation failures, analyse the edge cases that trigger them, improve the application's robustness, and rerun the tests against the corrected implementation.

2. Background

2.1 Fuzz Testing

Fuzz testing is a dynamic testing technique in which a program is supplied with random, unexpected, malformed, or boundary-value inputs. It helps reveal failures that may not be covered by a small set of manually selected test cases. Such failures can include unhandled exceptions, incorrect results, and weaknesses in input handling.

2.2 Hypothesis

Hypothesis is a property-based testing library for Python. Instead of requiring the tester to write every individual input, a test describes a property that should hold and Hypothesis generates examples to exercise that property. When a failure is found, Hypothesis attempts to reduce it to a simpler example that still reproduces the failure.

In this experiment, the tests use the @given decorator with st.text() | st.none() to generate string and None inputs. Each generated value is passed to one of the three target functions.

3. Bugs Identified

Function	Examples of problematic input	Issue exposed
units_consumed	None, empty string, ':'	Splitting and integer conversion can fail when two valid readings are not supplied.
daily_average	None, empty string	The input may not be a usable comma-separated list of integers; empty data can make averaging invalid.
tariff_rate	None, empty string, '-'	The category and slab may be missing, malformed, or unsupported.

These examples represent the types of edge cases exposed in the initial fuzz-testing runs. Hypothesis reported failing examples in the test output.

4. Execution Evidence

Figure 1. Original application execution

```
C:\Users\Admin\L6_L7_17_electricity_bill\Lab 7>python electricity_bill.py
==== Running units_consumed ====
Enter meter readings as previous:current (e.g. 1200:1350): 1200:1350
Units Consumed: 150

==== Running daily_average ====
Enter daily units as CSV (e.g. 5,7,6): 5,7,6
Daily Average: 6.0

==== Running tariff_rate ====
Enter a tariff code (e.g. DOM-A): DOM-A
Rate per Unit: 3
```

Figure 2. Initial failing fuzz tests – part 1

```
C:\Users\Admin\L6_L7_17_electricity_bill\Lab 7>python -m pytest test_electricity_bill.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.4, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Admin\L6_L7_17_electricity_bill\Lab 7
plugins: hypothesis-6.170.15
collected 3 items

test_electricity_bill.py::test_units_consumed_no_crash FAILED        [ 33%]
test_electricity_bill.py::test_daily_average_no_crash FAILED         [ 66%]
test_electricity_bill.py::test_tariff_rate_no_crash FAILED           [100%]

================================== FAILURES ===================================
_____________________ test_units_consumed_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\Admin\L6_L7_17_electricity_bill\Lab 7\test_electricity_bill.py", line 7, in test_units_consumed_no_crash
  |     def test_units_consumed_no_crash(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Admin\L6_L7_17_electricity_bill\Lab 7\test_electricity_bill.py", line 9, in test_units_consumed_no_crash
    |     units_consumed(s)
    |   File "C:\Users\Admin\L6_L7_17_electricity_bill\Lab 7\electricity_bill.py", line 6, in units_consumed
    |     return int(current) - int(previous)
    |            ^^^^^^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Admin\L6_L7_17_electricity_bill\Lab 7\test_electricity_bill.py", line 11, in test_units_consumed_no_crash
    |     assert False, f"units_consumed crashed with: {e}"
    | AssertionError: units_consumed crashed with: invalid literal for int() with base 10: ''
    | assert False
    | Failing test case: test_units_consumed_no_crash(
    |     s=':',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Admin\L6_L7_17_electricity_bill\Lab 7\test_electricity_bill.py", line 9, in test_units_consumed_no_crash
```

Figure 3. Initial failing fuzz tests – part 2

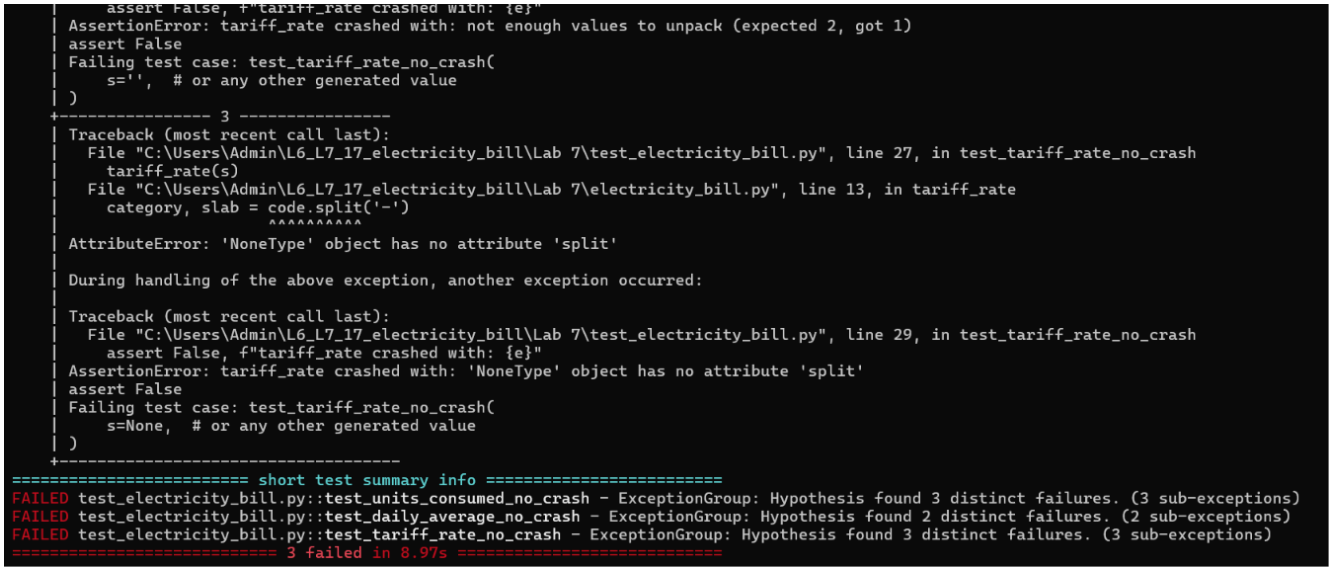


Figure 4. Corrected application execution

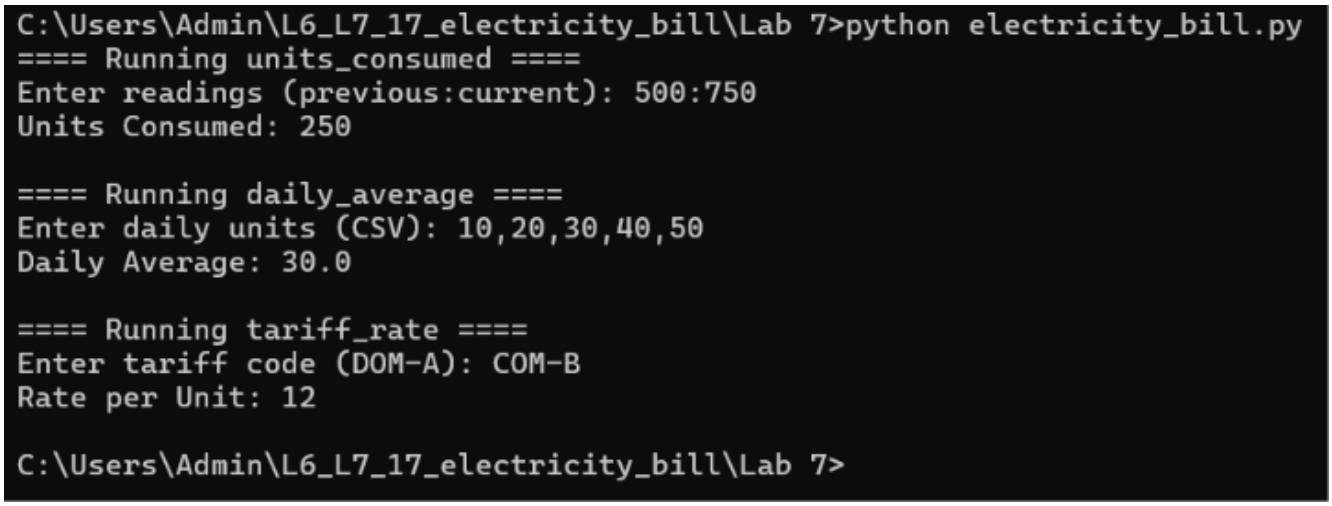
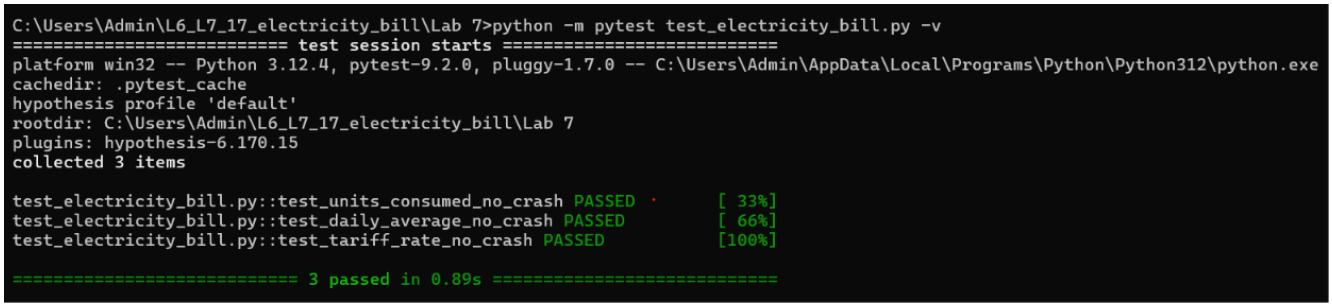


Figure 5. Final successful fuzz tests



5. Source Code

Appendix A – electricity_bill.py

Application source code

```
RATES = {"DOM": 3, "COM": 6}
SLAB = {"A": 1, "B": 2}



def units_consumed(reading):
    if not isinstance(reading, str):
        return None
    parts = reading.split(':')
    if len(parts) != 2:
        return None
    try:
        previous, current = map(int, parts)
        return current - previous
    except (ValueError, TypeError):
        return None



def daily_average(units_csv):
    if not isinstance(units_csv, str) or not units_csv.strip():
        return None
    try:
        units = [int(u.strip()) for u in units_csv.split(',')]
        return round(sum(units) / len(units), 2)
    except (ValueError, TypeError, ZeroDivisionError):
        return None
```

```
def tariff_rate(code):
    if not isinstance(code, str):
        return None
    parts = code.split('-')
    if len(parts) != 2:
        return None
    category, slab = parts
    try:
        return RATES[category.strip().upper()] * SLAB[slab.strip().upper()]
    except (KeyError, AttributeError):
        return None


def main():
    print("==== Running units_consumed ====")
    print("Units Consumed:", units_consumed(
        input("Enter readings (previous:current): ")))

    print("\n==== Running daily_average ====")
    print("Daily Average:", daily_average(
        input("Enter daily units (CSV): ")))

    print("\n==== Running tariff_rate ====")
    print("Rate per Unit:", tariff_rate(
        input("Enter tariff code (DOM-A): ")))


if __name__ == "__main__":
    main()
```

Appendix B – test_electricity_bill.py

Hypothesis test source code

```
from hypothesis import given, strategies as st
from electricity_bill import units_consumed, daily_average, tariff_rate

# Test 1: Units Consumed
@given(st.text() | st.none())
def test_units_consumed_no_crash(s):
    try:
        units_consumed(s)
    except Exception as e:
        assert False, f"units_consumed crashed with: {e}"



# Test 2: Daily Average
@given(st.text() | st.none())
def test_daily_average_no_crash(s):
    try:
        daily_average(s)
    except Exception as e:
        assert False, f"daily_average crashed with: {e}"

# Test 3: Tariff Rate
@given(st.text() | st.none())
def test_tariff_rate_no_crash(s):
    try:
        tariff_rate(s)
    except Exception as e:
        assert False, f"tariff_rate crashed with: {e}"
```

6. Repository and Submission Details

Individual (forked) repository: https://github.com/Anushka-Kumar/L6_L7_17_electricity_bill

Commit history: https://github.com/Anushka-Kumar/L6_L7_17_electricity_bill/commits/main/

SE Labs repository: https://github.com/Anushka-Kumar/SE-Lab-PES1UG24CS075

The Lab 7 folder has been added to the SE Labs repository. The source files, screenshots, and README and report have been committed and pushed, the link for the forked repository is also given (it is forked from the assigned repo and all the updates of codes are done there as well.)

7. Conclusion

This experiment demonstrated how property-based fuzz testing can expose input-validation weaknesses that may not appear during ordinary valid-input execution. Hypothesis generated unexpected inputs for all three application functions and helped identify cases that caused failures. After defensive input handling was implemented, the test suite was rerun and all three tests passed. The exercise highlights the importance of testing both expected and malformed inputs when building robust applications.